How retry timing actually works

This isn't about catching errors. It's about controlling exactly when the next attempt happens.

1. A naive retry calls the function again the instant it fails. Zero delay turns one outage into a thundering herd hitting the same broken endpoint.

2. **Exponential backoff** waits longer after each failure. The delay doubles: 1s, 2s, 4s, giving whatever broke real room to recover.

3. If every client backs off on the exact same schedule, retries synchronize into a storm. **Jitter** adds a small random offset so they spread out.

4. Backoff controls how long you wait. Jitter controls exactly when within that window. A real wrapper needs both.

> A retry wrapper that swallows the final error after retries run out is a wrapper that hides real outages from your logs.

The algorithm, boiled down

Six steps, from the first attempt to the moment it gives up or backs off.

<withRetry>

01 · ATTEMPT
Call fn() inside a try block

02 · SUCCESS
Return the result immediately, no retry logic runs

03 · FAILURE
catch(err) → check attempt against maxRetries

04 · EXHAUSTED
attempt ≥ maxRetries → throw the original error

05 · BACKOFF
delay = baseDelay * 2**attempt + jitter

06 · RETRY
await the delay, then call withRetry with attempt + 1

Wrap the function, not its result

withRetry()

withRetry.js

```
async function withRetry(fn, maxRetries = 3, baseDelay = 1000, attem
  try {
    return await fn();
  } catch (err) {

  }
}
```

> Takes the function itself, not its result, so it can be called again later without the caller doing anything.

> **attempt** starts at 0 and is internal bookkeeping. Callers never pass it themselves.

> If fn() succeeds, it returns immediately. No delay, no backoff math ever runs.

Decide when to actually give up

catch (err)

withRetry.js

```
if (attempt >= maxRetries) {
  throw err;
}
```

> Checked first, before any backoff math runs at all.

> Throwing, not swallowing, is what keeps this a wrapper instead of a silent failure.

> **attempt >= maxRetries** means every allowed attempt, including the first call, has already failed.

Double the wait, then shake it loose

delay + jitter

withRetry.js

```
const delay = baseDelay * 2 ** attempt + Math.random() * 300;
await sleep(delay);
```

> **2 ** attempt** doubles the wait every time: 1000ms, 2000ms, 4000ms for a 1000ms base.

> **Math.random() * 300** is the jitter. It keeps every client's retries from landing on the same millisecond.

> await sleep(delay) is the only place execution actually pauses. Everything before this runs synchronously.

One recursive call is the entire loop

withRetry()

withRetry.js

```
async function withRetry(fn, maxRetries = 3, baseDelay = 1000, attem
  try {
    return await fn();
  } catch (err) {
    if (attempt >= maxRetries) throw err;
    const delay = baseDelay * 2 ** attempt + Math.random() * 300;
    await sleep(delay);
    return withRetry(fn, maxRetries, baseDelay, attempt + 1);
  }
}
```

> The recursive call passes **attempt + 1**, so the next failure sees a longer delay and less runway.

> fn, maxRetries, and baseDelay stay constant across every call. Only attempt moves.

> This one line is the entire loop. No while, no manual counters outside the call stack.